# fil_033

# RFP Proposal: `Valist on NEAR`

**Name of Project:** `Valist`

**Link to RFP:** https://github.com/filecoin-project/devgrants/blob/master/rfps/near-and-filecoin.md

**RFP Category:** `app-dev`

**Proposer:** `@awantoch`

**Do you agree to open source all work you do on behalf of this RFP and dual-license under MIT and APACHE2 licenses?:** Yes

# Project Description

Valist enables web3-native software distribution (binaries, NPM packages, Docker images) by leveraging smart contract platforms and decentralized storage networks to coordinate access control and package name-spacing. It is modular, and supports swapping out both the smart contract and storage layers. The Valist client connects to designated networks, and provides backwards-compatible registry APIs for common tools such as NPM, Docker, and others. This means you can point your favorite package manager at Valist to resolve packages that are hosted on decentralized networks, or simply download an executable without relying on centralized upstream servers. It also provides a secure update framework for applications and infrastructure, as well as multi-sig for software releases. Currently, Valist uses EVM-based contracts for access control logic and package/namespace registration, and IPFS + Filecoin (via estuary) for storing data.

Our objective with this proposal is to build a NEAR package registry contract, and enable NEAR-native software distribution for various applications such as near-cli, nearup, near-api-js, and near-sdk-rs. For example, rather than pulling new NEAR releases from Github or S3 buckets, nearup a user can simply query the Valist registry contract on NEAR for new versions, and fetch them from IPFS/Filecoin – leveraging only web3 protocols from start to finish. NEAR maintainers (and if desired, community members) will be able to vote for new releases and govern distributions directly within the protocol. This further decentralizes the process for Nearcore Planned Updates.

This feature-set allows *any* software to be distributed with NEAR + IPFS/Filecoin, not just the core protocol tooling, so any developer in the ecosystem can leverage this for their packages.

## Deliverables

- A NEAR-native, AssemblyScript package registry smart contract that provides role-based access control, package namespacing, and versioning.
- A Golang SDK for the RPC APIs will be developed to enable Valist to support the protocol. This can become a `near-api-go` implementation, and has the potential to bring many Go developers into the NEAR ecosystem.
- Making the `nearcore` binaries available on Valist/IPFS/Filecoin. This requires some participation from the `nearcore` team to add the `valist publish` command to their release pipeline, or accept a PR with the changes.

- Updating NEAR documentation to include an option for installation via Valist/IPFS/Filecoin.

# Development Roadmap

## 1.) Development of registry contract

This deliverable will be a smart contract written in AssemblyScript to provide the following functionality:

- Creation of **organization/maintainer** accounts for software repositories. This will provide:
  - A namespace for the account that can be used to reference software within the organization.
  - Multiple organization-level accounts to maintain repositories within the organization, as well as the organization's metadata.
  - A multi-sig threshold that can be set for any organization-level action. This enables a system where no single organization admin can perform an action the rest disagree with. The threshold will be restricted to `N - 1`, where `N = total number of organization admin keys` to prevent lock-outs if a key is lost.
- Creation of **repositories** within these organization accounts. This will provide:
  - A namespace for the repository within the organization. At this level, the client can resolve `example-org/example-project` to find associated software releases.
  - An additional layer of access control, and enable election of developer roles.
  - A multi-sig threshold, similar to the organization-level but separately configurable on each repository.
- Creation of **releases** within each repository. This will provide:
  - An immutable version "tag" that can be used to reference each release.
  - At this level, releases can be resolved by `example-org/example-project/1.0.0`
  - A reference to a CID that contains the release data.
- Functions for managing organization and repository metadata, access control, and various view functions for querying data.

**Team:**

- Alec Wantoch - primary development
- Zachary Pelkey and Keenan Nemetz - support engineering and code review

**Milestone budget:** $50,000

**Timeline:** ~30 days (Split between ~Dec 1-15 and ~Jan 3 - Feb 2)

## 2.) Golang NEAR API client

This deliverable will create a Golang NEAR API client, similar/a port of `near-api-js`, and will provide the following functionality:

- Key management
- Functions that map to the NEAR JSON-RPC API
- Parity with `near-api-js` functionality where possible.

This will be integrated into the upstream `valist` codebase, which is MPL 2.0 licensed.

**Team:**

- Zachary Pelkey and Keenan Nemetz - primary development
- Alec Wantoch - support engineering and code review

**Milestone budget:** $100,000

**Timeline:** ~30 days (Split between ~Dec 1-15 and ~Jan 3 - Feb 2)

## 3.) Publishing `nearcore` binaries

This deliverable will create an example deployment pipeline for `nearcore`.

- Example script that contains the build steps for releasing `betanet` and `testnet` binaries.
- If applicable, a pull request to existing release pipeline. Otherwise, the example script can be used directly.
- Working with relevant NEAR protocol members on integrating this into the release process.

**Team:**

- Zachary Pelkey - primary development
- Keenan Nemetz and Alec Wantoch - support engineering and code review

**Milestone budget:** $5,000

**Timeline:** ~15 days (~Feb 3 - 18)

### 4.) Provide documentation for installing `nearcore` with `valist` as an alternative to `nearup`

This deliverable will add a page to the NEAR documentation for downloading and running `nearcore` binaries with Valist.

This page will be similar to the "Run a node with nearup" page.

A small modification will be made to the "Node Maintenance" page to include instructions for updating the nodes when installed through Valist.

**Team:**

- Zachary Pelkey and Keenan Nemetz - primary development
- Alec Wantoch - support engineering and code review

**Milestone budget:** $1,000

**Timeline:** ~15 days (~Feb 18 - March 5)

### 5.) (Optional) Modifying `nearup` to pull directly from web3 registry

This deliverable will create a fork of the `nearup` scripts that pull the `nearcore` binaries from the registry that can be used as the primary or secondary release pipeline.

This will require:

- Since `nearup` is written in Python, we will need to add the necessary read-only JSON-RPC API method(s) for fetching the latest releases from the NEAR contract.
- Adding a method for fetching the release binaries from `estuary`.

**Team:**

- Keenan Nemetz - primary development
- Zachary Pelkey and Alec Wantoch - support engineering and code review

**Milestone budget:** $10,000

**Timeline:** ~15 days (~Feb 18 - March 5)

## Total Budget Requested

Sum up the total requested budget across all milestones, and include that figure here. Ensure that it does not exceed the total funding limit on the RFP.

**Total:** $156,000 (without deliverable 5) or $166,000 (if deliverable 5 is accepted)

## Maintenance and Upgrade Plans

Our team relies on these protocols for our product development, and will continue to maintain these integrations for the long-term.

# Team

## Contact Info

## Team Members

- Alec Wantoch
  - Email: alec@valist.io
  - Github: https://github.com/awantoch
- Zachary Pelkey
  - Email: zach@valist.io
  - Github: https://github.com/jiyuu-jin
- Keenan Nemetz
  - Email: keenan@valist.io
  - Github: https://github.com/nasdf

## Team Member LinkedIn Profiles

- Alec's LinkedIn profile
- Zach's LinkedIn profile
- Keenan's LinkedIn profile

## Team Website

- https://valist.io

## Relevant Experience

Our team has strong experience in web3 software engineering, protocol implementations, and information security. We have worked with Protocol Labs closely on several initiatives, and are seeking to integrate with the NEAR ecosystem.

Each of our team members has prior experience in software engineering consulting, and have a long-term track record for contributing to open source.

Keenan - Keenan brings over a decade of full-stack software engineering, consulting, architecture design, and protocol development expertise. He has contributed to both the Ethereum and IPFS ecosystems. He has an affinity for protocol design and implementation, and has previously created an IPFS-powered git client, multiverse.

Zach - Zach is a full-stack distributed systems engineer with a background in cloud computing, and blockchain core development. He has previously helped launch an enterprise blockchain platform, from core node software to user-facing wallets. He is an expert at rapidly prototyping applications and refining them to production-grade quality.

Alec - Alec is a full-stack software engineer with a background in information security, blockchain core development, and web applications. He was previously the lead security engineer at Ethos prior to their acquisition by Voyager, where he helped build one of the first non-custodial universal wallet and blockchain API platforms. He specializes in blue-team security operations and development across the stack.

The Valist organization is building on top of these protocols, and will continue maintaining our integrations and contributing to the protocols themselves.

## Team code repositories

- Valist Repo
- Example Valist projects
- Ethereum light client
- Golang meta transaction library

# Additional Information

## Benefits to the NEAR ecosystem

- Valist brings web3-native software distribution to the NEAR ecosystem. This means that any software developed in or around NEAR can be distributed entirely by the protocols themselves, without having to rely on centralized infrastructure such as Github and AWS.
- As the NEAR ecosystem grows, Valist will serve as a unified place to discover software that's both relevant to the NEAR ecosystem, as well as web3 as a whole.
- Decentralized governance of software versions. As NEAR grows, the community can participate in the release process of node software, creating a more transparent and resilient release process that is self-servicing.
- Bringing Go developers into the NEAR ecosystem by providing Go-native tooling.
- Creating an easy, effective way for developers to use NEAR in their daily workflows.

## Benefits to the Filecoin ecosystem

- Software fits the exact type of data that the mission of Filecoin serves. Having useful data on Filecoin is essential to its future success, and the more software that is released through Valist, the more we can migrate away from centralized data stores, until eventually everything can stay web3-native from source code to built applications.
- Creating a retrieval market for data that is stored on Filecoin.
- Creating a discovery mechanism for finding software that's stored on Filecoin.
- Creating an easy, effective way for developers to use Filecoin in their daily workflows.